Django SMS Gateway System – Complete Full Notes

Technology: Python, Django, Django REST Framework, PostgreSQL, Celery, Redis

Project Levels:

1. Beginner – Simple Django SMS System
2. Intermediate – OTP and Notifications
3. Production – Celery + Redis
4. Advanced – Build Your Own SMS Gateway Platform

Table of Contents

1. Introduction to SMS Gateway
2. How SMS Gateway Works
3. SMS Gateway Providers
4. SMS Features
5. Project Architecture
6. Technology Stack
7. Project Setup
8. Project Folder Structure
9. Environment Variables
10. Database Models
11. SMS Provider Architecture
12. SMS Service Layer
13. OTP Generation System
14. OTP Verification API
15. User Registration SMS
16. Password Reset SMS
17. Order Notification SMS
18. SMS Logs
19. Celery + Redis Integration
20. Retry Mechanism
21. SMS API Endpoints
22. Build Your Own SMS System
23. Database Design
24. Security
25. Testing
26. Docker Deployment
27. Production Checklist
28. 30-Day Learning Roadmap
29. Interview Questions

---

1. Introduction to SMS Gateway

SMS Gateway হলো এমন একটি service/API যার মাধ্যমে Django application থেকে user-এর mobile number-এ SMS পাঠানো যায়।

SMS Gateway কেন ব্যবহার করা হয়?

- OTP Verification
- User Registration Notification
- Login Notification
- Password Reset
- Order Notification
- Payment Notification
- Delivery Notification
- Bulk SMS
- Promotional SMS

Example

একজন user registration করলে:

1. User registration form submit করবে।
2. Django user information validate করবে।
3. Django OTP generate করবে।
4. SMS Gateway API-তে request পাঠাবে।
5. SMS Provider SMS deliver করার চেষ্টা করবে।
6. User mobile-এ OTP পাবে।
7. User OTP verify করে account activate করবে।

---

2. How SMS Gateway Works

Architecture

```
User
 |
 v
Django Application
 |
 v
SMS Service Layer
 |
 v
SMS Gateway API
 |
 v
Mobile Network
 |
 v
User Mobile Phone
```

Working Process

Step 1: User Django API-তে request পাঠায়।

Step 2: Django request validate করে।

Step 3: SMS service message তৈরি করে।

Step 4: SMS provider-এর API call হয়।

Step 5: Provider request গ্রহণ করে এবং SMS পাঠানোর প্রক্রিয়া শুরু করে।

Step 6: Delivery report webhook-এর মাধ্যমে আসতে পারে।

---

3. SMS Gateway Providers

3.1 BulkSMSBD

Website: https://bulksmsbd.com/

Features:

- Bangladesh SMS
- Transactional SMS
- Promotional SMS
- OTP SMS
- API Integration

Use Case:

- Local Bangladesh applications
- E-commerce
- User verification

3.2 SSL Wireless

Website: https://www.sslwireless.com/

Features:

- SMS API
- OTP
- Transactional SMS
- Enterprise Messaging

Use Case:

- Business applications
- Banking-related notifications
- E-commerce notifications

3.3 Twilio

Website: https://www.twilio.com/

Features:

- International SMS
- OTP Verification
- WhatsApp API
- Voice API
- Global Communication

Use Case:

- International applications
- SaaS platforms
- Global OTP systems

3.4 Custom Provider

আমরা এমন architecture তৈরি করব যাতে ভবিষ্যতে provider পরিবর্তন করলেও মূল application পরিবর্তন করতে না হয়।

Provider Comparison

Provider	Best Use

BulkSMS BD	Bangladesh SMS
SSL Wireless	Bangladesh Business SMS
Twilio	International SMS
Custom	Flexible Architecture

Note: API URL, payload, authentication, sender ID এবং pricing provider-এর official documentation থেকে যাচাই করতে হবে।

---

4. SMS Features

4.1 OTP Verification

OTP = One Time Password

Use Cases:

- Registration Verification
- Login Verification
- Two-Factor Authentication
- Phone Number Verification
- Sensitive Action Verification

Example SMS:

```
Your verification code is 482913.
It will expire in 5 minutes.
```

OTP Security:

- 6-digit OTP
- Expiry time
- Maximum attempts
- Rate limiting
- One-time use
- OTP hashing
- Resend cooldown

4.2 Registration Notification

Example:

```
Welcome to our platform!
Your account has been created successfully.
```

4.3 Order Notification

Example:

```
Dear Customer,

Your order #ORD1024 has been confirmed.

Thank you for shopping with us.
```

Order SMS Types:

- Order Placed
- Order Confirmed
- Payment Successful
- Order Shipped
- Order Delivered
- Order Cancelled

4.4 Password Reset

Example:

```
Your password reset verification code is 928341.
Valid for 5 minutes.
```

4.5 Other Notifications

- Login Notification
- Payment Notification
- Appointment Reminder
- Delivery Notification
- Security Alert
- Admin Notification
- Promotional SMS

---

5. Project Architecture

Level 1: Beginner

Technology:

- Python
- Django
- Django REST Framework
- Requests
- SMS Provider API

Features:

- Send SMS
- OTP
- Registration SMS
- Password Reset SMS
- Basic SMS Logs

Level 2: Production

Technology:

- Django
- DRF
- PostgreSQL
- Celery
- Redis
- Docker

Features:

- Background SMS
- Retry Mechanism
- SMS Logs
- Delivery Reports
- Rate Limiting
- Multiple Providers

Level 3: Build My Own SMS System

Features:

- Multi-provider Support
- SMS API
- API Key Management
- Customer Management
- Wallet and Credits
- Bulk SMS
- Campaign Management
- Delivery Reports
- Analytics Dashboard

Note: নিজের SMS management platform তৈরি করা সম্ভব। তবে সরাসরি mobile network-এ SMS deliver করার জন্য authorized telecom operator বা SMS aggregator-এর connectivity প্রয়োজন।

---

6. Technology Stack

Technology	Purpose
Python	Programming
Django	Backend Framework
DRF	REST API
PostgreSQL	Database

Redis	Message Broker
Celery	Background Tasks
Requests	HTTP Requests
JWT	Authentication
Docker	Deployment
Pytest	Testing
Nginx	Reverse Proxy

---

7. Project Setup

Step 1: Create Project

```
mkdir sms_platform
cd sms_platform
```

Step 2: Create Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Linux:

```
source venv/bin/activate
```

Step 3: Install Packages

```
pip install django
pip install djangorestframework
pip install requests
pip install python-dotenv
pip install celery
pip install redis
pip install psycopg[binary]
pip install twilio
```

Step 4: Create Django Project

```
django-admin startproject config .
```

Step 5: Create Apps

```
python manage.py startapp sms
python manage.py startapp accounts
python manage.py startapp orders
python manage.py startapp notifications
```

Step 6: Run Server

```
python manage.py runserver
```

---

8. Project Folder Structure

```
sms_platform/
│
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── celery.py
│   └── wsgi.py
│
├── sms/
│   ├── migrations/
│   ├── providers/
│   │   ├── __init__.py
│   │   ├── base.py
│   │   ├── bulksmsbd.py
│   │   ├── sslwireless.py
│   │   └── twilio.py
│   │
│   ├── models.py
│   ├── services.py
│   ├── tasks.py
│   ├── serializers.py
│   ├── views.py
│   ├── urls.py
│   └── admin.py
│
├── accounts/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── urls.py
│   └── services.py
│
├── orders/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── services.py
│
├── notifications/
```

```
│   ├── models.py
│   ├── services.py
│   └── tasks.py
│
├── requirements.txt
├── .env
├── .gitignore
├── Dockerfile
├── docker-compose.yml
└── manage.py
```

---

9. Environment Variables

Create `.env`:

```
SECRET_KEY=your-secret-key
DEBUG=True

SMS_PROVIDER=bulksmsbd

BULKSMSBD_API_KEY=your-api-key
BULKSMSBD_SENDER_ID=your-sender-id
BULKSMSBD_API_URL=provider-api-url

SSLWIRELESS_API_KEY=your-api-key

TWILIO_ACCOUNT_SID=your-account-sid
TWILIO_AUTH_TOKEN=your-auth-token
TWILIO_PHONE_NUMBER=your-phone-number

REDIS_URL=redis://localhost:6379/0
```

Load Environment Variables:

```
# settings.py

import os
from dotenv import load_dotenv

load_dotenv()

SMS_PROVIDER = os.getenv("SMS_PROVIDER")

BULKSMSBD_API_KEY = os.getenv(
    "BULKSMSBD_API_KEY"
)

BULKSMSBD_SENDER_ID = os.getenv(
    "BULKSMSBD_SENDER_ID"
)

BULKSMSBD_API_URL = os.getenv(
    "BULKSMSBD_API_URL"
)
```

Important:

- Never upload `.env` to GitHub.
- Never hardcode API credentials.
- Use production environment secrets.

---

10. Database Models

10.1 SMSLog Model

প্রতিটি SMS request-এর information database-এ রাখবে।

```
# sms/models.py

from django.db import models


class SMSLog(models.Model):

    STATUS_CHOICES = [
        ("pending", "Pending"),
        ("sent", "Sent"),
        ("failed", "Failed"),
        ("delivered", "Delivered"),
    ]

    phone_number = models.CharField(
        max_length=20
    )

    message = models.TextField()

    provider = models.CharField(
        max_length=50
    )

    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default="pending"
    )

    provider_message_id = models.CharField(
        max_length=255,
        blank=True,
        null=True
    )

    error_message = models.TextField(
        blank=True,
        null=True
    )

    created_at = models.DateTimeField(
```

```
        auto_now_add=True
    )

    def __str__(self):
        return self.phone_number
```

10.2 OTPVerification Model

```
from django.db import models


class OTPVerification(models.Model):

    phone_number = models.CharField(
        max_length=20
    )

    otp_hash = models.CharField(
        max_length=128
    )

    expires_at = models.DateTimeField()

    is_verified = models.BooleanField(
        default=False
    )

    attempts = models.PositiveIntegerField(
        default=0
    )

    created_at = models.DateTimeField(
        auto_now_add=True
    )
```

10.3 Additional Models

Advanced project-এর জন্য:

- SMSProvider
- ProviderConfiguration
- APIKey
- SMSCampaign
- CampaignRecipient
- Wallet
- Transaction
- DeliveryReport

---

11. SMS Provider Architecture

আমরা provider-independent architecture তৈরি করব।

11.1 Base Provider

```
# sms/providers/base.py

from abc import ABC, abstractmethod


class BaseSMSProvider(ABC):

    @abstractmethod
    def send_sms(self, phone_number, message):
        pass
```

11.2 BulkSMSBD Provider

নিচের code architecture-এর example। Actual endpoint এবং payload provider-এর documentation অনুযায়ী configure করতে হবে।

```
# sms/providers/bulksmsbd.py

import requests

from django.conf import settings

from .base import BaseSMSProvider


class BulkSMSBDProvider(BaseSMSProvider):

    def send_sms(self, phone_number, message):

        payload = {
            "api_key": settings.BULKSMSBD_API_KEY,
            "senderid": settings.BULKSMSBD_SENDER_ID,
            "number": phone_number,
            "message": message,
        }

        response = requests.post(
            settings.BULKSMSBD_API_URL,
            data=payload,
            timeout=15
        )

        response.raise_for_status()

        return response.json()
```

11.3 Twilio Provider

Install:

```
pip install twilio
```

Code:

```
# sms/providers/twilio.py
```

```
from twilio.rest import Client

from django.conf import settings

from .base import BaseSMSProvider


class TwilioProvider(BaseSMSProvider):

    def __init__(self):

        self.client = Client(
            settings.TWILIO_ACCOUNT_SID,
            settings.TWILIO_AUTH_TOKEN
        )

    def send_sms(self, phone_number, message):

        result = self.client.messages.create(
            body=message,
            from_=settings.TWILIO_PHONE_NUMBER,
            to=phone_number
        )

        return {
            "message_id": result.sid,
            "status": result.status
        }
```

11.4 SSL Wireless Provider

```
# sms/providers/sslwireless.py

from .base import BaseSMSProvider


class SSLWirelessProvider(BaseSMSProvider):

    def send_sms(self, phone_number, message):

        # Implement using official SSL Wireless API.
        # Add authentication, payload, timeout,
        # response validation and error handling.

        raise NotImplementedError(
            "Configure SSL Wireless API"
        )
```

---

12. SMS Service Layer

এখন provider নির্বাচন করার জন্য একটি service তৈরি করব।

```
# sms/services.py
```

```
from django.conf import settings

from .providers.bulksmsbd import BulkSMSBDProvider
from .providers.twilio import TwilioProvider
from .providers.sslwireless import SSLWirelessProvider


def get_sms_provider():

    provider = settings.SMS_PROVIDER

    providers = {
        "bulksmsbd": BulkSMSBDProvider,
        "twilio": TwilioProvider,
        "sslwireless": SSLWirelessProvider,
    }

    provider_class = providers.get(provider)

    if not provider_class:
        raise ValueError(
            "Unsupported SMS provider"
        )

    return provider_class()


def send_sms(phone_number, message):

    provider = get_sms_provider()

    return provider.send_sms(
        phone_number,
        message
    )
```

Benefits:

- Easy provider switching
- Clean architecture
- Reusable service
- Maintainable code
- Easy testing

---

13. OTP Generation System

Python-এর `secrets` module ব্যবহার করব।

```
import secrets


def generate_otp():

    return str(
```

```
        secrets.randbelow(900000) + 100000
    )
```

Example:

```
482913
```

OTP SMS Function

```
def send_otp_sms(phone_number, otp):

    message = (
        f"Your verification code is {otp}. "
        "Valid for 5 minutes."
    )

    return send_sms(
        phone_number,
        message
    )
```

OTP Hashing

```
import hashlib
import hmac


def hash_otp(otp, secret):

    return hmac.new(
        secret.encode(),
        otp.encode(),
        hashlib.sha256
    ).hexdigest()


def verify_otp_hash(otp, stored_hash, secret):

    generated_hash = hash_otp(
        otp,
        secret
    )

    return hmac.compare_digest(
        generated_hash,
        stored_hash
    )
```

Use a dedicated OTP pepper stored in environment variables, not the Django SECRET_KEY by default.

---

14. OTP Verification API

Endpoint:

```
POST /api/accounts/verify-otp/
```

Request:

```
{
    "phone_number": "88017XXXXXXXX",
    "otp": "482913"
}
```

Expected Response:

```
{
    "success": true,
    "message": "OTP verified successfully"
}
```

Verification Logic:

1. Phone number check.
2. OTP record retrieve.
3. Expiry check.
4. Maximum attempt check.
5. OTP hash comparison.
6. Mark as verified.
7. Prevent OTP reuse.

Important:

- Use database transactions.
- Increment attempts atomically.
- Add resend cooldown.
- Apply per-phone and per-IP rate limits.
- Never log plaintext OTP.

---

15. User Registration SMS

Registration Flow:

```
User
 |
 v
Registration API
 |
 v
Validate Information
 |
 v
Create Inactive User
 |
 v
Generate OTP
 |
 v
Save OTP Hash
 |
 v
Send SMS
```

```
 |
 v
Verify OTP
 |
 v
Activate User
```

Endpoint:

```
POST /api/accounts/register/
POST /api/accounts/verify-otp/
```

Example Request:

```
{
    "username": "faruk",
    "phone_number": "88017XXXXXXXX",
    "password": "StrongPassword123"
}
```

Example Response:

```
{
    "success": true,
    "message": "OTP sent successfully"
}
```

---

16. Password Reset SMS

Flow:

```
User
 |
 v
Enter Phone Number
 |
 v
Generate OTP
 |
 v
Send SMS
 |
 v
Verify OTP
 |
 v
Set New Password
```

Endpoints:

```
POST /api/accounts/password-reset/
POST /api/accounts/password-reset/verify/
POST /api/accounts/password-reset/confirm/
```

Example SMS:

```
Your password reset code is 928341.
Valid for 5 minutes.
```

Security:

- Do not reveal whether an account exists.
- Limit requests.
- Expire reset tokens.
- Invalidate token after successful reset.
- Use Django password validation.

---

17. Order Notification SMS

Order events:

- Order Created
- Order Confirmed
- Payment Successful
- Order Shipped
- Order Delivered
- Order Cancelled

Example:

```
def send_order_notification(
    phone_number,
    order_id
):

    message = (
        f"Your order #{order_id} "
        "has been confirmed."
    )

    return send_sms(
        phone_number,
        message
    )
```

Production system-এ order status change হওয়ার পর notification task dispatch করবে। Database transaction সফল হওয়ার আগে SMS পাঠাবে না।

---

18. SMS Logs

SMS logs-এর কাজ:

- Track SMS requests.
- Track provider responses.
- Store message IDs.
- Record errors.
- Monitor delivery status.
- Generate reports.

Status:

```
PENDING
SENT
FAILED
DELIVERED
UNDELIVERED
```

Note: Provider accepted status মানেই SMS delivered নয়। Delivery confirmation-এর জন্য provider webhook বা delivery report প্রয়োজন।

---

19. Celery + Redis Integration

Celery background task processing-এর জন্য ব্যবহার করা হয়।

Redis message broker হিসেবে কাজ করতে পারে।

Architecture

```
Django API
    |
    v
Celery Task
    |
    v
Redis Broker
    |
    v
Celery Worker
    |
    v
SMS Provider
    |
    v
Mobile Phone
```

19.1 Install

```
pip install celery redis
```

19.2 Create celery.py

```
# config/celery.py

import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)

app = Celery("config")

app.config_from_object(
    "django.conf:settings",
    namespace="CELERY"
```

```
)

app.autodiscover_tasks()
```

19.3 Update config/init.py

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

19.4 Configure Redis

```
# settings.py

CELERY_BROKER_URL = (
    "redis://localhost:6379/0"
)

CELERY_ACCEPT_CONTENT = ["json"]

CELERY_TASK_SERIALIZER = "json"

CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Dhaka"
```

19.5 Create Celery Task

```
# sms/tasks.py

from celery import shared_task

from .services import send_sms


@shared_task(
    bind=True,
    max_retries=3,
    default_retry_delay=10
)
def send_sms_task(
    self,
    phone_number,
    message
):

    try:

        return send_sms(
            phone_number,
            message
        )

    except Exception as exc:
```

```
        raise self.retry(
            exc=exc
        )
```

19.6 Call Celery Task

```
from sms.tasks import send_sms_task

send_sms_task.delay(
    "88017XXXXXXXX",
    "Your order has been confirmed."
)
```

19.7 Start Redis

```
redis-server
```

19.8 Start Celery Worker

Windows development:

```
celery -A config worker -l info -P solo
```

Linux:

```
celery -A config worker -l info
```

---

20. Retry Mechanism

SMS provider temporary failure হলে retry করতে হবে।

Example:

```
@shared_task(
    bind=True,
    max_retries=5
)
def send_sms_task(
    self,
    phone_number,
    message
):

    try:

        return send_sms(
            phone_number,
            message
        )

    except Exception as exc:

        raise self.retry(
            exc=exc,
            countdown=30
        )
```

Production improvements:

- Exponential backoff
- Maximum retry limit
- Provider timeout
- Idempotency key
- Failed task monitoring
- Dead-letter handling
- Duplicate prevention

Important: Provider timeout হলে SMS ইতোমধ্যে পাঠানো হয়ে থাকতে পারে। তাই retry করার আগে idempotency এবং provider message status বিবেচনা করতে হবে।

---

21. SMS API Endpoints

Method	Endpoint	Purpose
POST	/api/accounts/register/	Registration
POST	/api/accounts/verify-otp/	OTP Verification
POST	/api/accounts/resend-otp/	Resend OTP
POST	/api/accounts/login/	Login
POST	/api/accounts/password-reset/	Password Reset
POST	/api/sms/send/	Send SMS
GET	/api/sms/logs/	SMS History
GET	/api/sms/logs/{id}/	SMS Details
POST	/api/orders/	Create Order
POST	/api/orders/{id}/notif	Order Notificatio

	y/	n

---

22. Build Your Own SMS System

এখন আমরা একটি complete SMS Gateway Management Platform তৈরি করব।

Main Modules

1. Authentication
2. User Management
3. API Key Management
4. Provider Management
5. SMS Management
6. OTP Management
7. Bulk SMS
8. Campaign Management
9. Wallet Management
10. Credit Management
11. Transaction History
12. Delivery Reports
13. Analytics Dashboard
14. Admin Dashboard

Advanced Features

- Multiple SMS Providers
- Provider Fallback
- Country-Based Routing
- SMS Scheduling
- Bulk SMS Campaign
- API Authentication
- Customer Credits
- Wallet Transactions
- Delivery Webhooks
- SMS Usage Analytics

---

23. Database Design

Main Models

```
User
  |
  |---- APIKey
  |
  |---- Wallet
  |
  |---- Transaction
  |
  |---- SMSLog
  |
```

```
  |---- SMSCampaign
  |
  |---- OTPVerification

SMSProvider
  |
  |---- ProviderConfiguration
  |
  |---- SMSLog

SMSCampaign
  |
  |---- CampaignRecipient
```

Model Responsibilities

Model	Responsibility
User	Customer information
APIKey	API authentication
Wallet	Customer balance
Transaction	Credit history
SMSProvider	Provider information
ProviderConfiguration	Provider settings
SMSLog	SMS history
OTPVerification	OTP management
SMSCampaign	Campaign management

Campaign Recipient	Campaign recipients

---

24. Security

Production security checklist:

- Use HTTPS.
- Keep API credentials in environment variables.
- Hash OTP before storing.
- Set OTP expiry.
- Limit OTP attempts.
- Apply rate limiting.
- Validate phone numbers.
- Protect API endpoints.
- Use authentication.
- Use role-based permissions.
- Avoid logging OTP.
- Prevent duplicate SMS.
- Verify webhook signatures.
- Restrict admin access.
- Add audit logs.
- Implement user consent and opt-out.
- Follow applicable Bangladesh SMS and sender ID regulations.

---

25. Testing

Postman দিয়ে API testing করতে পারো।

Test Cases

Test	Expected Result
Valid phone number	SMS request accepted
Invalid phone number	Validation error
Correct OTP	Verified
Incorrect OTP	Verification failed

Expired OTP	Expired error
Too many OTP requests	Rate limited
Provider failure	Retry / failed status
Duplicate request	No duplicate SMS
Delivery webhook	Status updated

Unit Testing Example

```
from django.test import TestCase

from unittest.mock import patch

from sms.services import send_sms


class SMSTestCase(TestCase):

    @patch("sms.services.get_sms_provider")
    def test_send_sms(self, mock_provider):

        mock_provider.return_value.send_sms.return_value = {
            "status": "success"
        }

        result = send_sms(
            "88017XXXXXXXX",
            "Test Message"
        )

        self.assertEqual(
            result["status"],
            "success"
        )
```

---

26. Docker Deployment

Production deployment-এর জন্য Docker ব্যবহার করা যায়।

Services:

- Django

- PostgreSQL
- Redis
- Celery Worker
- Nginx

Example docker-compose structure:

```
services:

  web:
    build: .
    command: gunicorn config.wsgi:application
    depends_on:
      - db
      - redis

  db:
    image: postgres:16

  redis:
    image: redis:7

  celery:
    build: .
    command: celery -A config worker -l info
    depends_on:
      - redis
      - db
```

Production-এ database credentials, ports, volumes, health checks, environment variables এবং secure networking configure করতে হবে।

---

27. Production Checklist

- Django project configured
- PostgreSQL configured
- SMS provider integrated
- OTP generation implemented
- OTP verification implemented
- Registration notification implemented
- Password reset implemented
- Order notification implemented
- SMS logs implemented
- Celery configured
- Redis configured
- Retry mechanism implemented
- Rate limiting implemented
- API authentication implemented
- Multiple providers supported
- Webhook implemented
- Unit tests written

- Docker configured
- Production deployment completed

---

28. 30-Day Learning Roadmap

Days	Topics	Target
1–3	Django Setup, DRF	Basic API
4–6	SMS Provider Integration	Send SMS
7–9	OTP System	OTP Verification
10–12	Registration, Password Reset	Authentication
13–15	SMS Logs, Database	SMS History
16–18	Celery + Redis	Background SMS
19–21	Retry, Rate Limiting	Reliability
22–24	Multiple Providers	Provider Switching
25–27	Bulk SMS, Campaigns	Advanced SMS
28–30	Testing, Docker, Deployment	Production Project

---

29. Interview Questions

Basic Questions

1. What is an SMS Gateway?
2. What is an SMS API?
3. How does SMS delivery work?
4. What is OTP?
5. Why do we use SMS verification?
6. What is the difference between transactional and promotional SMS?
7. What is an SMS provider?

Django Questions

1. How do you integrate SMS API with Django?
2. What is a Django service layer?
3. Why should we separate provider logic?
4. How do you store SMS logs?
5. How do you handle provider errors?
6. How do you implement OTP verification?
7. How do you secure API credentials?

Celery Questions

1. What is Celery?
2. Why do we use Redis?
3. What is a background task?
4. What is a Celery worker?
5. How do you implement retry?
6. How do you prevent duplicate tasks?
7. What is exponential backoff?

Advanced Questions

1. How do you implement multiple SMS providers?
2. How do you implement provider fallback?
3. How do you implement rate limiting?
4. How do you implement bulk SMS?
5. How do you track SMS delivery?
6. How do you design an SMS SaaS platform?
7. How do you implement wallet and credit management?
8. How do you secure a public SMS API?

Final Recommendation

এই project তিনটি ধাপে complete করবে।

Phase 1: Beginner

- Django
- DRF
- BulkSMSBD
- SMS API
- OTP

- Registration Notification

Phase 2: Production

- PostgreSQL
- Celery
- Redis
- SMS Logs
- Retry Mechanism
- Rate Limiting
- Delivery Reports

Phase 3: Advanced

- Multiple Providers
- SMS Gateway API
- API Key Management
- Wallet System
- Bulk SMS
- Campaign Management
- Analytics Dashboard

Final Goal: Build a complete, secure, scalable Django SMS Gateway Management Platform.

Important: এই notes হলো learning roadmap এবং architecture guide। Production deployment-এর আগে provider-specific API integration, error handling, security, idempotency, delivery reporting এবং automated testing সম্পূর্ণ করতে হবে.